Cloud computing(AWS)

1. What is Cloud Computing?

Cloud Computing is using the internet to store, manages and process data, instead of using you own computer's hard drive.

- You don't need to buy or manage physical servers.
- Pay-as-you-go model: Pay only for what you use.

2. What is Cloud?

Cloud is the internet-based storage and services system that lets you access data or program online, anytime and anywhere.

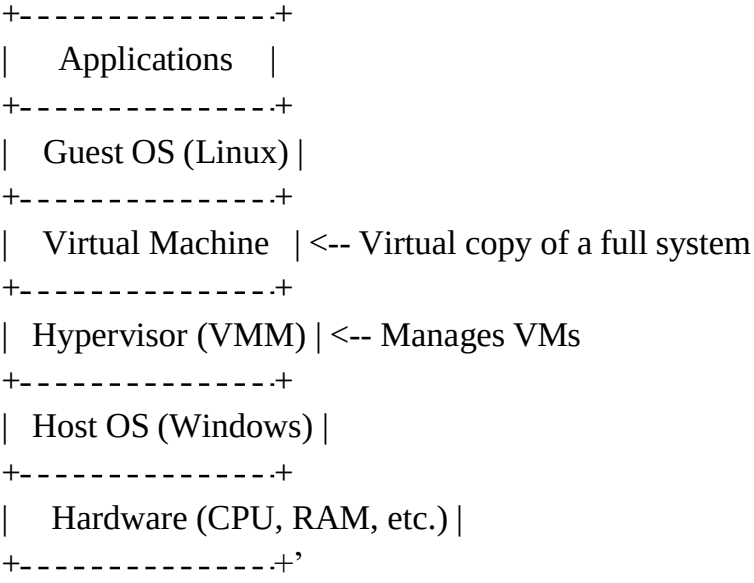
Cloud= **Virtualization** + **Automation** + **Scalability + Accessibility via Internet**
Cloud computing uses **virtualization** to derivers **on-demand services** via the Internet.

3. What is Virtualization?

Virtualization is the process of creating a virtual version of something – like a computer, server or storage – using software.

- Imagine 1 laptop running 2 or 3 different OS (windows, linux, etc.) at the same time.
- Each OS runs in its own virtual machine (VM).

4. Components of Virtualization?

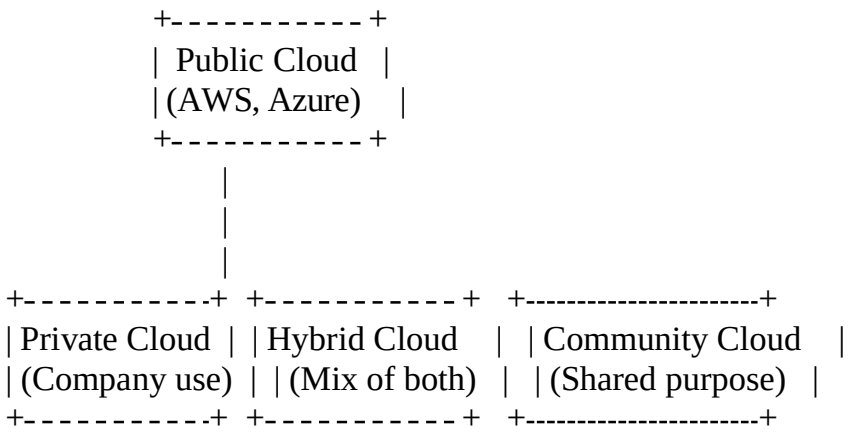


- **VM (Virtual Machine):** A virtual computer (runs its own os). A software – based emulation of physical components.
- **Hypervisor:** Software that creates and manages virtual machines.
  - **Type 1 (Bare- metal):** Runs directed on hardware.
  - **Type 2(Hosted):** Runs on a host OS.
- **Host OS:** The original OS on your physical machines.
- **Guest OS:** Operating systems inside a VM.

5. Traditional Computer vs Virtualization

Feature	Traditional Computer	Virtualization
Usage	One OS only	Multiple OS
Hardware Use	Less efficient	Highly efficient
Cost	High	Low
Flexibility	Low	High
Isolation	No	Yes
Management	Manual	Centralized
Setup Time	Longer	Faster
Testing Environments	Difficult	Easy
Scalability	Limited	Scalable
Example	Personal PC	VMware / VirtualBox

6. **Cloud Computing Deployment Models:**



   - **Public Cloud:** Owned & manages by third – party provided. Pay as you go.
   - **Private Cloud:** Used by a single organization more secure and controlled.
   - **Hybrid Cloud:** Mix of public + private cloud.
7. **Cloud Computing Services Models:**
   - **IAAS (Infrastructure as a Services):** Provides virtual hardware like server, storage, and networks.
     Ex: Amazon EC2 - You rent virtual machine to run your apps.
   - **PAAS (Platform as a Service):** Provides a platform with tools to builds and run application without managing the hardware.
     Ex: Google App Engine – You just write codes; Google runs it.
   - **SAAS (Software as a Service):** Provides ready-to-use software over the internet.
     Ex: Gmail/Google Docs – just open and use in a browser.

Differences Between IaaS, PaaS, SaaS

Feature	IaaS	PaaS	SaaS

|----------|----------|----------|----------|

| **What you get** | Virtual hardware | Tools to build apps | Complete software |

| **User control** | Full control | App control only | Just use the app |

| **User manages** | OS, app, data | App, data | Nothing (just use it) |

| **Example** | Amazon EC2 | Google App Engine | Gmail, Dropbox |

| **Use case** | Host servers | Build apps | Use apps |

What is EC2?

EC2 (Elastic Computer Cloud) is a service by **Amazon web Services (AWS)** that lets you run virtual servers (called instances) on the cloud that provided:

- **Scalability & resizable** compute capacity in the cloud.
- Allows you to run application on virtual machines without investing in physical hardware.
  - **Features:**
    - Pay -as-you-go billing.
    - Choose OS (Linux/windows)
    - Flexible instance types (t2.micro, etc.)
    - Can attach storage, IP, security groups

Why SSH Was Created?

Before SSH, remote servers were accessed using protocols like:

- **Telenet**
- **Rlogin**
- **FTP (for file transfer)**

  These protocols were **unsecured**, meaning:

  - Data, passwords, and commands were sent in **plain text** over the network.
  - Any attacker using a packet sniffer(like Wireshark) could easily capture credentials.
  - SSH (Secure Shell) is a protocol used to access and manage EC2 instance on AWS.
    port number =22
  - RDP(Remote Desktop Protocol) to a Microsoft protocol that allows you to _ remote access and controls window computer or services .
    port number = 3389

❖ How to create EC2 Instance in AWS:

- Step 1: Go to ec2 service
- Step 2: click on instance
- Step 3: click on launch instance with name
- Step 4: select O.S images
- Step 5: Instance type selects free_tier.
- Step 6: Create now key pair (.pem)
- Step 7: Network setting allows SSH Traffic http & https.
- Step 8: check number of instance '1'.
- Step 9: launch Instance
- Step 10: select instance & connect.
- Step 11: Ec2 instance connect click connect.

❖ Step to create IAM user: -

- Step 1: Log in to AWS console.
- Step 2: search for IAM services.
- Step 3: Click on user in IAM.
- Step 4: Click on create user.
- Step 5: Give username & select I want to create an I am user.
- Step 6: password set -> next
- Step 7: Attach policy to given admins access click created user.
- Step 8: Download CSV file where user credential are stored.

❖ EBS Volume (Elastic Block Store) :

- EBS Volume is a durable (store data without loose over time) , high-performance block storage device that you can attach to EC2 instance.
- They are virtual hard drives, you can format , mount.

EBS Volume Types:

1. SSD [Solid State Drive]:
   - These are fast storage volume used in AWS designed for quick and frequent reading & writing of data.
   - They are good when you need high speed and performance.
   - They are best for running database hosting web pages etc.
   - General purpose SSD (gp2/gp3).
   - Provisional IOPS SSD (io1/io2).

2. HDD [Hard Disk Drive]:
   - There is storage volume in AWS that are good for handling large files.
   - They are slower than SSD.
   - Cold HDD (st1).
   - Throughout HDD (c1).

3. Magnetic Volume:
    - There are data alder types of storage in AWS based on magnetics hard disk drivers.
    - Good for very small workload that don't need speed.

Features	SSD	HHD
Speed	Fast	Slow
Latency	Low	High
Cost	More expensive	Cheaper
Durability	More reliable	Less reliable
Use	Web App etc.	Bigdata backup

❖ How to Create Volume:

- Step 1: Elastic Block Store
- Step 2: click on volume
- Step 3: click on create volume
- Step 4: In volume setting
  volume type: General purpose SSD (gp3)
  size (GIB): - 50GB
  * up to 30GB is free

  Availability zone:- select location of instance
- Step 5: click on create volume

To attach volume to instance

- Step 6: Select volume created
- Step 7: click on Actions
- Step 8: click on Attach volume:
  1. select the instance
  2. select deula name: /dev/xvdbb
  3. click on attach volume

To make filesystem

- Step 9: List information of about block device: lsblk
- Step 10: Display disk space usage of file system: df -h
- Step 11: Use file system attach to virtual disk.
  mkfs. ext4 /dev/<device name>
  mkfs. ext4 -L mydata /dev/xvdf
  Note:
  File Systems:- File system is way an operating system organizes, store and manages data on a storage devices.

File system types are:

- ext4
- xfs
- ext3
- ext2
- FAT32
- ZFS
- NTFS
- nfs

➢ Step 12: Disk attach to folder for use:
mount /dev/<device name> /mnt

➢ Step 13: Display disk usages along with file system
df -hT

➢ Step 14: unmount used to safely disconnect a mounted file system or storage device from system.
unmount /dev/<device-name>

✓ After attaching disk to folder for use:

1. mount /dev/<device-name> /folders-name
2. cd /mnt
3. ls
4. touch aws.txt
5. mkdir cloud

Note: - To verify that our data is present in disk.

6. cd
7. unmount /dev/<device-name>

❖ Disk Partition: process of diving a storages device into smaller logical units called partitions. Each partition can be:

- Formatted with a file system.
- Mounted separately.
- Used for different purpose.

Note: If our external storage devices: 30GB to divide it.

➢ Step 1: In amazon linux if we have external storage of name (nvm1n1).
➢ Step 2: To check external storage name: lsblk.
➢ Step 3: To create view, modify or delete partitions on a disk drive:

- Sudo fdisk /dev/<device-name>
- Help -> m press
- Press -> n
- Default p: enter
- Port number: enter
- First sector: enter
- Last sector stops: +10G

- Step 4: click on create partition:
  - help: m
  - save: w
- Step 5: Reload update partition into kernel to make recognize now partition without rebooting after using fdisk: parprode lsblk
- Step 6: To mount partition
  - Sudo mkdir /dev/nvme1n1p1
  - Sudo mkdir /mnt/mydata
  - Sudo mount /dev/nvme1n1   /mydata
- Step 7: To unmount partition

  unmount   /dev/nvme1n1p1

  - Temporary mount: - Attaching a storages devices or partition to a directory only for current session. Mount will disappear when of server. In before example we have done temporary mount.
  - Permanent mount: Means attaching a disk or partition in such a way that it remains mounted automatically every time the system rebots.

    Step to do permeant mount:
    - df -hT
    - lsblk
    - open configuration file in linux that list disk partitions & devices to be mounted automatically at boot time.
    - sudo mkfs.ext4 /dev/xvdf – format the volume
    - sudo mkdir /mnt/ebs – create a mount points.
    - sudo mount   /dev/xvdf   /mnt/ebs – mount the volume temporarily
    - sudo vim /etc/fstab – edit etc/fstab for permanent mount & then added uuid
    - sudo unmount /mnt/ebs – Test the configuration & unmount the volume
    - sudo unmount -a – Remount all filesystems specified
    - df -hT – to check it works.

  - Snapshots: Backup copy of your hard disk. Save the current state of your volume. To create new volume or name data to another instance we can use:
    - Steps to create example:
      - Step 1: Click on create snapchats
      - Step 2: Select volume id
      - Step 3: click on create snapshot
    - We can use our created snapshot to create Volume
      - Step 1: select snapshot
      - Step 2: click Action -> create volume from snapshot
    - To add snapshot to instance
      - Step 1: Click on create (launch) instance
      - Step 2: configure storage -> Advance volume click

- Step 3: device name size , snapshot select
- Click on launch instance

- **AMI (AMAZON MACHINE IMAGE): -**
  - Step 1: touch aws.txt
  - Step 2: adduser <username>
  - Step 3: select action
  - Step 4: select image & template
  - Step 5: creating Image click
  - Step 6: give image name
  - Step 7: Reboot instance ☐ doesn't click
  - Step 8: click on create images

Note: Do not terminate server until it available.

- **Security Group [Firewall]:** A network security system that monitor and controls incoming & outcoming network traffic based on predefined security rules. Preventing unauthorized access and malicious traffic from entering or leaving the network.
  Steps to create security groups:
  - Step 1: Network & security click
  - Step 2: select security groups
  - Step 3: click on create security groups
  - Step 4: Fill basic details:
    - security group name
    - description: allows to access
  - Step 5: select inbound Rule & click on Add rules:

Type	Source
SSH	Anywhere
HTTP	Anywhere

  - Step 6: click on create security groups.
  - Step 7: now security group is created use it's on an ec2 instance.
  -

- **Instance Type:** In a Virtual machine that runs on Amazon EC2.

1. General Purpose:
   - Provided a balance of computer, memory, and networking power.
   - This instance is good for hosting websites, running web server, handling small apps, managing code repositors like Git, running development or testing environment.
   - There is t & m series instance.

2. Compute Optimized:
   - Computer optimized instance are virtual machines in AWS that are designed to give you very powerful CPU's (processor).

- There are best when your app needs to do lot of calculations, fast processing, or intense computing.
- There is c services instance.

3. Memory Optimized:
   - Memory optimized instance are designed to deliver fast performance for workloads process large data sets in memory.
   - This is r series instance

4. Accelerated Optimized:
   - Accelerated computing instance are special virtual machine that have extra hardware to do complex tasks faster than regular CPU.
   - This is p series instance.

5. Storage Optimized:
   - Are specials AWS Virtual machine that are built to handle very large amount of data store on local disks and allows for fast reading and writing of that data.
   - This is i series instance.

❖ **Purchasing Options:**

1. On Demand Instance: you pay only for the seconds that your on-demand instance is in running state with a 60 second minimum.
2. Reserved Instance: Are a way to save money when you know you will be using a virtual machine for a long time.
3. Dedicated Hosts: Physical server with instance capacity fully dedicated to your use.
4. Dedicated instance: EC2 instance that run on physical servers fully dedicated to you not shared with AWS other customer.
5. Spot instance : Unused instance offered by AWS at up to 90% discount compared to on-demand prices.

❖ **EFS (Elastic File Systems):**

✓ **What is EFS (Elastic File Systems):**

- Amazon EFS is a cloud-based file storages service.
- It allows multiple EC2 instance to share file in real-time like a shared folders in a local networks.
- EFS uses NFS (Network file System) protocol to access file.

Common networks ports:

Service	Port
SSH	22
HTTP	80
HTTPS	443
RDP	3389
DNS	53
MYSQL	3306

NFS	2049
Jenkins/Tomcat	8080

✓ **EFS Setup and Mount (Using Amazon Linux):**

- Step 1: Install EFS Utils:
  sudo yum install -y amazon-efs-utils
- Step 2: Create Mount Directory:
  mkdir efs
  sudo chmod 777 efs
- Step 3: Mount the EFS File System: Get your EFS ID and AWS region (like fs-123456789 and ap-south-1) from AWS Console.
  sudo mount -t efs -o tls fs-12345678:/ efs
- Step 4: If using mount targets via IP:
  sudo mount -t nfs4 -o nfserver=4.1 10.0.0.100:/ efs
- Step 5: Use EFS (create Files , Share Between EC2):
  cd efs
  sudo touch backup.txt
  echo "My EFS test file" > backup.txt
  ls – l

🛠 Storage Type Comparison

Storage Type	Description	Examples
DAS	Direct attached, local	EC2 instance store, SSD
NAS	Shared over network	EFS, NFS, Samba
SAN	Enterprise block-level	iSCSI, Fibre Channel

❖ EFS & NFS:

⌘ Steps to Attach EFS to EC2:

Prerequisites:

- EC2 running in same region & vpc.
- EFS with mount target in same subnets AZ as EC2.

Step-by-Step:

1. **Install NFS client**
   - sudo apt update -y
   - sudo apt install nfs-common -y

2. **Get Mount IP/DNS**
   from AWS Console → EFS → Attach → Mount via IP → copy IP (eg.,,172.31.x.x:/)
3. **Create mount directory:**
   sudo mkdir -p /mnt/efs

4. **Check connectivity:**
   ping 172.31.x.x # If blocked, update SG

5. **Manual Mount**
   sudo mount -t nfs 172.31.x.x /mnt/efs

6. **Make Mount Persistent**
   sudo vim /etc/fstab
   Add line:
   172.31.x.x:/ /mnt/efs nfs4 default netdev 0 0

7. **Reload & mount:**
   sudo systemctl restart servers
   sudo mount -a

8. **Unmount if needed**
   sudo unmount /mnt/efs

🔒 **Security Group Rules for EFS**

- Types: customer TCP
- Port: 2049
- Source: EC2 SG or VPC CIDR (eg., 173.31.0.0/16)

🆚 EBS vs EFS

Feature	EBS	EFS
Type	Block Storage	Network File Storage (NFS)
Attachment	One EC2 (same AZ)	Multiple EC2s (cross-AZ)
Access Mode	Like a hard disk	Like a shared folder
Scalability	Manual resize	Automatic, elastic
Performance	Lower latency	Slightly higher latency
Pricing	Cheaper	Slightly more expensive

AWS VPC Networking – Full Notes

1. **What is Networking?**
   Networking refers to the practice of connecting computer and other devices to share data and resources.

   Key Terms:

   - **Node:** Any device connected to a network.
   - **IP Address:** A unique identifier for each device on a network.
   - **Subnet:** A smaller division of a network.

2. **IP Addressing Basics**
   - IPV4 is a 32-bits address written in dotted decimal (eg., 192.168.1.1).
   - Composed of 4 octets (each 8 bits), each ranging from 0 to 255.

     Example: 255.255.255.255 is the highest possible IP.

3. **IP Address Classes**

Class	Range	Default Subnet	Mask NO.of Host	Use Case
A	1-26	255.0.0.0 (/8)	16 Millions	Large network
B	128-191	255.255.0.0 (/16)	65,000	Medium networks
C	192-223	255.255.255.0 (/24)	254	Small networks
D	224-239	N/A	N/A	Multicast
E	240-255	N/A	N/A	Research

Note: 127.x.x.x is reserved for loopback (localhost)

4. **Private IP Ranges**

   Used within internal/private networks:

Class	Private Ranges
A	10.0.0.0 – 10.255.255.255.255
B	172.16.0.0 – 172.31.255.255
C	192.168.0.0 – 192.168.255.255

These are not routable over the internet.

5. CIDR Notation (Classes Inter-Domain Routing)

- Example: 192.168.1.0/24
- "/24" means first 24 bits are network portion → 2^8 = 256 total IPs - 2 = **254 usable IPs**

Common CIDR Values:

CIDR	Subnet Mask	Usable IPs
/8	255.0.0.0	16,777,214
/16	255.255.0.0	65,534
/24	255.255.255.0	254
/30	255.255.255.252	2
/32	255.255.255.255	1 (host only)

6. How to Calculate CIDR IP Ranges

Example: 192.168.10.0/28

- 32 - 28 = 4 bits for host
- 2^4 = 16 IPs total
- Subtract 2 (network + broadcast) → **14 usable Ips**

IP Range:

- Network: 192.168.10.0
- Usable: 192.168.10.1 to 192.168.10.14
- Broadcast: 192.168.10.15

What are AWS VPC?

Amazon Virtual Private Cloud (VPC) is a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define.

You control:

- IP address ranges
- Subnets
- Route tables
- Internet Gateway
- NAT Gateway
- Networks ACL & security groups

VPC Core Concepts

1. CIDR Block

- Defines IP range of VPC (e.g., 10.0.0.0/16)
- Min: /28 (16 IPs), Max: /16 (65,536 IPs)

2. Subnet

- Public Subnet → Has Internet Gateway
- Private Subnet → No direct internet access
- Subnet must be in one AZ

3. Route Table

- Controls how traffic is directed
- Each subnet must be associated with one route table

4. Internet Gateway (IGW)

- Enables public internet access
- Must be attached to the VPC

5. NAT Gateway

- Allows private subnets to access internet (outbound only)
- Must reside in a public subnet

6. Security Group

- Instance-level **stateful** firewall
- Allows inbound/outbound traffic

7. Network ACL

- Subnet-level **stateless** firewall
- Can allow or deny traffic explicitly

Example VPC Design

Component	CIDR Block	AZ	Notes
VPC	10.0.0.0/16	-	65,536 IPs
Public Subnet	10.0.1.0/24	us-east-1a	Internet-facing subnet
Private Subnet	10.0.2.0/24	us-east-1b	Internal services subnet

5. NACL – Network ACL

What is it?
A network-level firewall that controls traffic to/from subnets.

- Works at the **subnet level** (not instance)
- **Stateless**: You must define rules for both inbound and outbound separately

Feature	NACL	Security Group
Scope	Subnet-level	Instance-level
Type	**Stateless**	**Stateful**
Rules	Allow & Deny	Only Allow
Order	Rules evaluated in numbered order (top → bottom)	All rules are evaluated

🔹 **How it works:**

- Each VPC has a default NACL that allows all traffic
- Custom NACLs can be created and associated with subnets
- Useful for adding **deny rules**, which security groups don't support

---

6. Security Groups

What is it?
A virtual firewall attached to EC2 instances. Controls **inbound and outbound** traffic at the **instance level**.

🔹 **Key Points:**

- **Stateful**: Allowing inbound automatically allows return traffic outbound
- Rules are based on:
  - Protocol (TCP, UDP, etc.)
  - Port Range (e.g., 22, 80, 443)
  - Source/Destination (CIDR/IP/another SG)

---

7. Placement Groups

What is it?
Placement Groups help control how EC2 instances are placed inside AWS infrastructure for performance and availability.

🔹 **Why use Placement Groups?**

- Improve **latency and bandwidth** between EC2s
- Ensure **fault tolerance** or **isolation**

Types of Placement Groups

Type	AZ Scope	Speed	Fault Tolerance	Use Case
Cluster	Same AZ	Very High	Low	HPC, gaming, big data
Spread	Across AZs	Normal	Very High	Critical small workloads
Partition	Same AZ	High	Medium–High	Kafka, Hadoop, Cassandra

🔹 **How to Create:**

1. EC2 Dashboard → Placement Groups → Create
2. Give it a name
3. Choose placement strategy: Cluster / Spread / Partition
4. When launching EC2 → Advanced Details → Select Placement Group

🔹 **Important Notes:**

- All EC2s in the group must be in **same region**
- Cluster: compatible instance types required
- You **can't move existing EC2s** into placement groups

🔹 **Interview Tip:**

- Q: Why use Cluster PG? → Low-latency communication
- Q: Spread vs Partition? → Spread = 1 per rack, Partition = group per rack

---

8. Elastic IP (EIP)

What is it?
A **static public IPv4** address provided by AWS. Unlike normal public IPs, EIPs don't change when the instance is stopped/restarted.

🔹 **Use Cases:**

- Need a fixed IP for DNS, firewall, or SSH access
- Quickly remap to new EC2 in case of failure

🔹 **How to Use:**

1. EC2 Dashboard → Elastic IPs → Allocate Elastic IP
2. Click Actions → Associate Elastic IP
3. Choose the EC2 instance and private IP

🔹 **Important Points:**

- 1 EIP per account by default (can increase via request)
- **Free only when attached to a running instance**
- Charges ~~$0.005/hr if unused (~~₹9–10/day)
- EIPs work **only with IPv4**

Why Use EIPs in Real-Time?

Scenario	Why EIP Helps
SSH access	Fixed IP avoids daily updates
App whitelisted in firewalls	Fixed IP required
Replace failed EC2 with another	EIP reassociation = instant recovery
DNS A record	IP stability ensured

---

9. NIC – Network Interface Card (ENI)

What is it?

- NIC connects EC2 to a network (VPC)
- In AWS, called **Elastic Network Interface (ENI)**

🔹 **Key Characteristics:**

- Every EC2 has a **primary ENI** (can't be detached)
- Can attach/detach **secondary ENIs** on supported instance types

ENI Contains:

- Private IP (one or more)
- Public IP (if assigned)
- MAC address
- Security Group

Default ENI Limits by Instance Type:

Instance Type	Max ENIs	IPv4 per ENI	Total IPv4 Addresses
t2.micro	2	2	4
t3.small	3	4	12
t3.medium	3	6	18
m5.large	3	10	30
c5.2xlarge	4	15	60
r5.4xlarge	8	30	240

🔹 How to Check ENI Limit:

1. AWS Docs → ENI limits
2. AWS CLI:

```
aws ec2 describe-instance-types \
  --instance-types t3.medium \
  --query "InstanceTypes[*].NetworkInfo.MaximumNetworkInterfaces"
```

Why Use ENIs in Real-Time?

Reason	Benefit
Failover	Quick recovery with same IPs
Multi-Networking	Connect to multiple subnets/VPCs
Security Zoning	Use different SGs for different traffic
Traffic Segments	Separate ENIs for separate services
Advanced Use	Required for firewall, VPN, IDS systems

Real-Time VPC Architecture for Frontend, Backend, Database & Environment Management

Where Frontend, Backend, and Database are Stored in VPC:

✅ Typical Architecture Inside a VPC:

Layer	Placement	Example Services	Public/Private
Frontend	Public Subnet	Web Servers (React, Angular, HTML static hosted via NGINX/Apache/EC2)	Public
Backend	Private Subnet	APIs, Application Servers (Spring Boot, Node.js, Django)	Private
Database	Private Subnet	MySQL, PostgreSQL, MongoDB (Managed via RDS or EC2)	Private

Example VPC Diagram:

```
VPC (10.0.0.0/16)
 ├── Public Subnet (10.0.1.0/24)
 │    ├── ALB (Application Load Balancer)
 │    ├── EC2 for Frontend (Optional)
 │    └── IGW (Internet Gateway)
 └── Private Subnet (10.0.2.0/24)
      ├── Backend EC2 (Spring Boot APIs)
      ├── RDS Database (MySQL/PostgreSQL)
      └── NAT Gateway (Outbound internet for backend)
```

Traffic Flow:

- **User → ALB (Public Subnet) → Backend API (Private Subnet) → DB (Private Subnet)**
- Backend has **no public IP**, only goes out via **NAT Gateway** if needed (e.g., software updates).
- **Database is fully private**, only backend servers can access it.

Best Practices in Real-Time:

✔ Separate VPC per Environment (Recommended for Enterprises):

- `VPC-Dev`: For developers, experiments.
- `VPC-Test`: For testing and QA.
- `VPC-Prod`: For live customers.

🟩 Benefit:

- Strong isolation. If something crashes in Dev, Prod isn't affected.

OR Single VPC with Separate Subnets (Used in Startups/Small Projects):

Environment	Subnet Example
Dev	10.0.1.0/24
Test	10.0.2.0/24
Prod	10.0.3.0/24

OR Use Multiple AWS Accounts (Large Scale Best Practice):

- `Account 1`: Dev
- `Account 2`: Test
- `Account 3`: Production

🟩 Managed via AWS Organizations, SCP (Service Control Policies), and AWS Control Tower.

Security Measures in Real-Time:

- 🟩 Frontend → Public Subnet (with ALB or CloudFront).
- 🟩 Backend → Private Subnet (no public IP).
- 🟩 Database → Strictly private.
- 🟩 NACLs and Security Groups tightly control what talks to what.

Step-by-Step: Custom VPC Setup

Part 1: Create Custom VPC

1. AWS Console → Search "VPC"
2. Click "Create VPC"
3. Name: MyCustomVPC
4. IPv4 CIDR: 10.0.0.0/16
5. Tenancy: Default → Create VPC

Part 2: Create Subnets

Public Subnet

- Go to Subnets → Create Subnet
- Name: PublicSubnet
- VPC: MyCustomVPC
- AZ: ap-south-1a
- CIDR: 10.0.1.0/24

Private Subnet

- Repeat above steps
- Name: PrivateSubnet
- AZ: ap-south-1a
- CIDR: 10.0.2.0/24

Part 3: Internet Access

Internet Gateway

- Go to IGW → Create IGW → Name: MyIGW → Attach to MyCustomVPC

Public Route Table

- Go to Route Tables → Create → Name: PublicRouteTable
- Add Route: 0.0.0.0/0 → Target: MyIGW
- Associate PublicSubnet to this Route Table

Part 4: Launch EC2 Instances

Public EC2 Instance

- Launch → Name: PublicEC2
- AMI: Amazon Linux 2
- Subnet: PublicSubnet
- Public IP: Enable
- SG: Allow SSH from My IP
- Key Pair: Use/create new one

Private EC2 Instance

- Launch → Name: PrivateEC2
- Subnet: PrivateSubnet

- Public IP: Disable
- SG: Allow SSH from PublicEC2's private IP

What is difference between security Group & NACL (Network Access Control List). (IMP)

Features	Security Group(SG)	NACL(Network ACL)
Applied to	**Instance level** (EC2, RDS, etc.)	**Subnet level** (affects all instances inside the subnet)
Type	**Stateful** – If you allow incoming traffic, the return traffic is automatically allowed.	**Stateless** – You must allow both incoming and outgoing traffic separately.
Default Rule	All **deny** by default until you allow rules.	All **allow** by default until you block them.
Rules	Only **allow** rules (cannot explicitly deny traffic).	**Allow** or **Deny** rules are possible.
Use Case	Fine-grained control for specific instances.	Broad control for the whole subnet (acts like a firewall).

What is NACL?

- NACL = Network Access Control List.
- Works at the subnet level to allow or block specific traffic based on rules.
- Stateless: You must explicitly allow inbound and outbound separately.
- Rule number order: Lower rule number gets evaluated first, and the first match wins.
- Used for an extra layer of security along with Security Groups.
- **Example use case:**
  You want to block a specific IP from accessing **any instance** in a subnet → Use NACL.

What is AWS Networking Terms?

✓ Endpoints:

- Allow you to connect to AWS services (like S3, DynamoDB) **privately** without going through the internet.
- Types:
  - **Interface Endpoint** (uses private IPs inside VPC)
  - **Gateway Endpoint** (for S3 and DynamoDB)

✓ VPN (Virtual Private Network):

- Secure tunnel between **your on-premise network** and AWS VPC.
- Can be **Site-to-Site VPN** or **Client VPN**.

✓ **DHCP (Dynamic Host Configuration Protocol)**

- Assigns **IP addresses automatically** to instances in your VPC.
- AWS uses **DHCP option sets** to configure domain names, DNS servers, etc.

What is EC2 Placement Groups?

Placement Groups = How AWS arranges your EC2 instances physically in the data center.

Types:

1. **Cluster** – All instances in the same rack & AZ → Low latency, high performance (but low fault tolerance).
2. **Spread -** Instances spread across multiple racks → High fault tolerance (if one rack fails, others are safe).
3. **Partition** – Instances split into **partitions**, each with its own rack → Used in large distributed systems like Hadoop.

How to created Step-by-step: Create NACL in AWS?

1. Go to **VPC Dashboard → Network ACLs → Create NACL**.
2. Select **VPC** and give it a **name**.
3. Add **Inbound Rules** (traffic coming in).
4. Add **Outbound Rules** (traffic going out).
5. **Associate** the NACL with one or more subnets.

What is Load Balance in AWS & its types?

AWS Load Balancers = Distribute traffic to multiple EC2 instances.

Types:

1. Application Load Balancer (ALB):
    - Works at **Layer 7 (HTTP/HTTPS)**.
    - Routes traffic based on URL, hostname, path.
    - Best for **web applications**.
2. Network Load Balancer (NLB):
    - Works at **Layer 4 (TCP/UDP)**.
    - Handles **millions of requests per second** with ultra-low latency.
    - Best for **gaming, real-time apps**.
3. Gateway Load Balancer (GLB):
    - Routes traffic to **third-party virtual appliances** (firewalls, intrusion detection).

What is difference between ALB & NLB?

Feature	ALB	NLB
OSI Layer	Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP)
Traffic type	Web requests	Any TCP/UDP traffic
Routing	Host-based, Path-based	Port-based only
Performance	High	Extreme
Latency	Slightly higher	Ultra-low
Static IP	➕ No (uses DNS)	✅ Yes
Preserve Client IP	➕ (uses X-Forwarded-For)	✅ Yes
Use case	Websites, APIs, microservices	Gaming,chat,streaming.

AWS Load Balancing & Auto Scaling - Detailed Notes

1. ELB – Elastic Load Balancer

What is ELB?
Elastic Load Balancer (ELB) automatically distributes incoming traffic across multiple targets (like EC2s) to ensure high availability, fault tolerance, and better scalability. It provides a single point of access to your application and integrates with Auto Scaling Groups.

Why Use Load Balancers?

- Distribute load across multiple downstream instances
- Single point of access to your application
- Handles instance failures and reroutes traffic
- Performs regular health checks
- Increases availability across multiple AZs.
- Integrates with Auto Scaling Groups.

Types of Load Balancers

Type	Description
Application LB	Works at Layer 7 (HTTP/HTTPS), supports path- and host-based routing
Network LB	Works at Layer 4 (TCP/UDP), ultra-low latency and high-performance
Gateway LB	Designed for third-party virtual appliances (firewalls, intrusion detection)
Classic LB	Legacy option (Layer 4/7), not recommended for new apps

---

Key Concepts of ELB

- **Target Group**: A group of registered targets (like EC2 instances)
- **Listeners**: A process that checks for connection requests using configured protocol/port
- **Health Checks**: Monitors instance health; ELB stops routing traffic to unhealthy targets
- **Cross-Zone Load Balancing**: Ensures even traffic distribution across AZs
- **Sticky Session**: Also known as session affinity, uses cookies to bind a user to the same EC2 instance

---

Application vs Gateway Load Balancer

Feature	Application Load Balancer (ALB)	Gateway Load Balancer (GLB)
OSI Layer	Layer 7 (Application Layer – HTTP/HTTPS)	Layer 3/4 (Network/Transport Layer – IP, TCP/UDP)
Purpose	Distribute web traffic to web servers or microservices	Route traffic to third-party **security appliances** like firewalls
Use Case	Web apps, REST APIs, host/path-based routing	Security inspection, deep packet inspection, threat detection
Target Type	EC2 instances, containers, Lambda	Appliance VMs (security tools)

Feature	Application Load Balancer (ALB)	Gateway Load Balancer (GLB)
Protocol Support	HTTP, HTTPS	Any TCP/UDP protocol
Routing Features	URL-based routing, host-based routing, redirect, fixed response	Transparent traffic forwarding (no changes to packets)
Health Checks	At HTTP level (can check paths like /health)	At network level (based on target's response)
Ideal For	Web applications, microservices, API Gateway backend	Security-focused architecture (firewall, IDS/IPS, antivirus)

Summary:

- Use **ALB** when you need smart web traffic routing (like domain/path-based).
- Use **GLB** when you want to **insert security appliances** transparently between client and your VPC.

Step-by-Step: Create Application Load Balancer

1. Go to EC2 Dashboard → Load Balancers → Create Load Balancer
2. Choose **Application Load Balancer**
3. Name your ALB
4. Select VPC and availability zones
5. Add listener (e.g., HTTP on port 80)
6. Configure security group (allow port 80)
7. Create/choose a **target group**
8. Register your EC2 instances with this target group
9. Review and **create ALB**

When to Use Each Type:

Type	Use Case
Application LB	Web apps, microservices, routing based on URL/path/hostname
Network LB	Games, real-time services, low-latency TCP traffic
Gateway LB	Advanced security appliances, third-party virtual appliances
Classic LB	Only for legacy compatibility (not for new architectures)

ELB Features

- SSL termination
- Sticky sessions (session affinity)
- Autoscaling integration
- Monitoring via CloudWatch
- High availability by default

---

2. Auto Scaling Groups (ASG)

What is Auto Scaling?
Auto Scaling helps you automatically adjust the number of EC2 instances based on load, cost, or performance.

Key Concepts

Term	Explanation
Launch Template	Defines AMI, instance type, key pair, SG, etc.
Auto Scaling Group	Manages EC2 lifecycle: launch, replace, terminate
Scaling Policy	Defines how and when to scale in/out (manual, scheduled, dynamic)
Health Check	ASG replaces unhealthy instances
Cooldown Period	Time buffer before another scale action

Benefits of Auto Scaling

- **High availability**
- **Cost-effective**: Run only the required number of instances
- **Fault-tolerant**: Automatically replaces failed instances
- **Handles spikes**: Automatically scales during traffic peaks

---

Step-by-Step: Create Auto Scaling Group

1. Go to EC2 → Auto Scaling Groups → Create
2. Select or create a **Launch Template**
3. Choose the VPC and subnets
4. Attach an existing **Application Load Balancer** (optional)
5. Set **minimum, desired, and maximum capacity**
6. Configure **health checks** and **grace period**
7. Add **scaling policies**:
    - Target tracking (e.g., maintain 50% CPU)
    - Step scaling (threshold-based)
    - Scheduled actions
8. Add **notifications** (optional)
9. Review and **create**

    OR

    1. Launch Template & Configuration
    2. Create Auto Scaling Group
    3. Select VPC and Subnet
    4. Attach Load Balancer (option)
    5. Configure Scaling Policies

6. Health Checks
7. Add Notification (option)
8. Review & Create

Auto Scaling Policies

Policy Type	Use Case
Manual	User manually changes instance count
Scheduled	Scale at fixed times (e.g., night, weekdays)
Target Tracking	Keep a metric (e.g., CPU) at a target value
Step Scaling	Scale in/out by specific steps based on thresholds

---

Real-Time Example Scenario

You run an e-commerce site.

- Use **Application Load Balancer** to distribute user traffic
- Attach it to an **Auto Scaling Group**
- Set target tracking policy to keep CPU usage ~60%
- Minimum instances = 2, Max = 6
- If traffic spikes, ASG adds more EC2s and balances via ALB

Monitoring ELB & ASG

- **CloudWatch Metrics**
  - ELB: Healthy host count, request count, latency
  - ASG: Group desired/actual size, instance launch/terminate count
- **Alarms**: Create CloudWatch alarms for CPU, traffic, failures
- **Logging**: Enable access logs for ELB to S3

Summary: ELB vs ASG

Feature	Elastic Load Balancer	Auto Scaling Group
Purpose	Distribute traffic	Manage instance count
Works At	Request/Connection Level	Infrastructure Level
Key Benefit	Load distribution + availability	High availability + cost control
Type	ALB / NLB / Gateway LB / Classic	Dynamic, Scheduled, Manual Scaling

Let me know if you want a diagram or real project-based deployment using ELB + ASG!

AWS IAM (Identity and Access Management) – Complete Guide

□ What is IAM?

- **IAM (Identity and Access Management)** is an AWS service that helps you **securely manage access to AWS services and resources**.
- It allows you to create and manage:
  - **Users**
  - **Groups**
  - **Roles**
  - **Policies**
- IAM controls **who can do what** in your AWS account.

□ Why IAM is Needed?

- Security is a top priority in cloud.
- IAM enables:
  - **Granular permission control** (who can access what and how).
  - **Principle of Least Privilege** — users only get the permissions they need.
  - Avoids sharing **root account credentials**, which is dangerous.

□ Key Components of IAM

Component	Description
Users	Individual accounts for people/services.
Groups	Collections of users with shared permissions.
Roles	Temporary access for AWS services or external users.
Policies	Documents that define permissions (JSON-based).

✅Detailed Explanation of Components

□ IAM Users

- Represents a **person or application**.
- Can have:
  - Username and Password (AWS Console Access).
  - Access Keys (Programmatic Access via CLI/SDK/API).
- Example: nilesh_dev_user

□ IAM Groups

- Collection of users.
- Permissions assigned to a group apply to all users in the group.

- Example:
  - Group: Developers
  - Users: nilesh_dev_user, john_dev_user

IAM Roles

- **No username/password.**
- Provides **temporary permissions**.
- Used by:
-
  - AWS services (e.g., EC2 accessing S3).
  - Other AWS accounts (cross-account access).
  - External identity providers (e.g., Google, Facebook, SAML).

Example: An EC2 instance uses an **Instance Role** to access an S3 bucket without storing credentials.

IAM Policies

- Written in **JSON format**.
- Defines **allow or deny actions** for resources.
- Two types:
  - **AWS Managed Policies:** Predefined by AWS.
  - **Customer Managed Policies:** Created by you.
- Example Policy:

```
{
 "Version": "2012-10-17",
 "Statement": [{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
 }]
}
```

➡ This allows **all S3 actions** on **all resources**.

How IAM Works Internally?

Authentication:

- IAM verifies **who you are** (using passwords, keys, etc.).

Authorization:

- IAM checks **what you are allowed to do** (based on policies).

IAM Policy Syntax Breakdown

Key	Meaning
Version	Policy language version. Use "2012-10-17" (latest).

Key	Meaning
Effect	"Allow" or "Deny".
Action	AWS service actions (s3:PutObject, ec2:StartInstances, etc.).
Resource	Specifies which AWS resources (arn:aws:s3:::my-bucket/*).
Condition	(Optional) Adds conditions (IpAddress, MFA, etc.).

□ Types of IAM Access

Access Type	Usage
AWS Management Console	Username + Password (GUI).
Programmatic Access	Access Key ID + Secret Access Key (CLI/SDK/API).

□ Root User Warning

- The **root account** has **unrestricted access**.
- Never use root for daily tasks.
- Use root only for:
  - First-time setup.
  - Billing.
  - Changing account settings.

□ Security Best Practices in IAM

1. **Avoid using the root account.**
2. **Enable MFA (Multi-Factor Authentication)** for root and IAM users.
3. Use **groups** to manage permissions.
4. Apply **least privilege principle** — give only required permissions.
5. Rotate **access keys** regularly.
6. Monitor with **IAM Access Analyzer** and **CloudTrail**.

□ Example Scenario:

▸ Goal:

- Allow an EC2 instance to read/write from an S3 bucket.

▸ Steps:

1. Create an **IAM Role** with the following policy:

```
json
CopyEdit
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:*",
    "Resource": "arn:aws:s3:::mybucket/*"
```

```
  }]
}
```

2. Attach the role to EC2 instance.
3. EC2 can now access the bucket **without any key/password**.

□ Important AWS Managed Policies Examples:

Policy Name	Description
AmazonS3FullAccess	Full access to S3.
AmazonEC2FullAccess	Full access to EC2.
AdministratorAccess	Full access to all AWS services. (□ Use carefully)
AmazonEC2InstanceConnect	Required for EC2 Instance Connect feature.

□ Diagram: IAM Working Flow

```
pgsql
CopyEdit
+------------+     +---------+     +-------------+
|  User/Role | ---> |  Policy | ---> | AWS Service |
+------------+     +---------+     +-------------+
```

□ Summary in One Line:

□ **IAM is the security backbone of AWS that manages who can access which services and resources, and under what conditions.**

🔑 AWS IAM (Identity and Access Management) – Step by Step

IAM helps you securely control access to AWS services and resources.

Step 1: Sign in to AWS Management Console

- Log in as the Root user (the account you created when signing up for AWS).
  ⚠ Best practice: Don't use the Root account for daily work, only for account setup.

Step 2: Create an IAM User

1. Go to IAM from AWS console.
2. In the sidebar → Click Users → Add users.
3. Enter a username (e.g., developer1).
4. Select Access type:

- Password (AWS Management Console access) → If user needs web login.
- Access key (programmatic access) → If user needs CLI/SDK access.

👉 Example: Create developer1 with both Console + CLI access.

Step 3: Assign Permissions

- AWS gives 3 options:
  1. Add user to group (recommended).
  2. Copy permissions from existing user.
  3. Attach policies directly.

👉 Example:

- Create a group named Developers.
- Attach AWS managed policy AmazonS3FullAccess to the group.
- Add user developer1 into Developers group.
  ✅ Now, developer1 can access S3 fully.

Step 4: Generate Security Credentials

- If you chose CLI access → AWS generates Access Key ID + Secret Access Key.
- Download the .csv file (very important, secret key is shown only once).
- If Console access → Set a custom password or let AWS auto-generate one.

👉 Example:
developer1 now has:

- Username: developer1
- Password: xxxxxx (for console login)
- Access Key + Secret Key (for CLI/SDK login)

Step 5: Apply MFA (Multi-Factor Authentication)

1. In IAM → Select developer1.
2. Go to Security credentials → Assign MFA device.
3. Choose Virtual MFA (like Google Authenticator).

4. Scan QR → Enter codes → Done.

👉 Example: Even if developer1 password leaks, hacker needs MFA code.

Step 6: Create IAM Policies (Fine-Grained Access Control)

- A Policy is a JSON document defining permissions.

👉 Example: Create a policy to allow read-only access to S3:

json

CopyEdit

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": "*"
    }
  ]
}
```

- Attach this policy to a group ReadOnlyUsers.

Step 7: Use Roles (for EC2, Lambda, etc.)

- Instead of giving Access Keys to apps, use IAM Roles.

👉 Example:

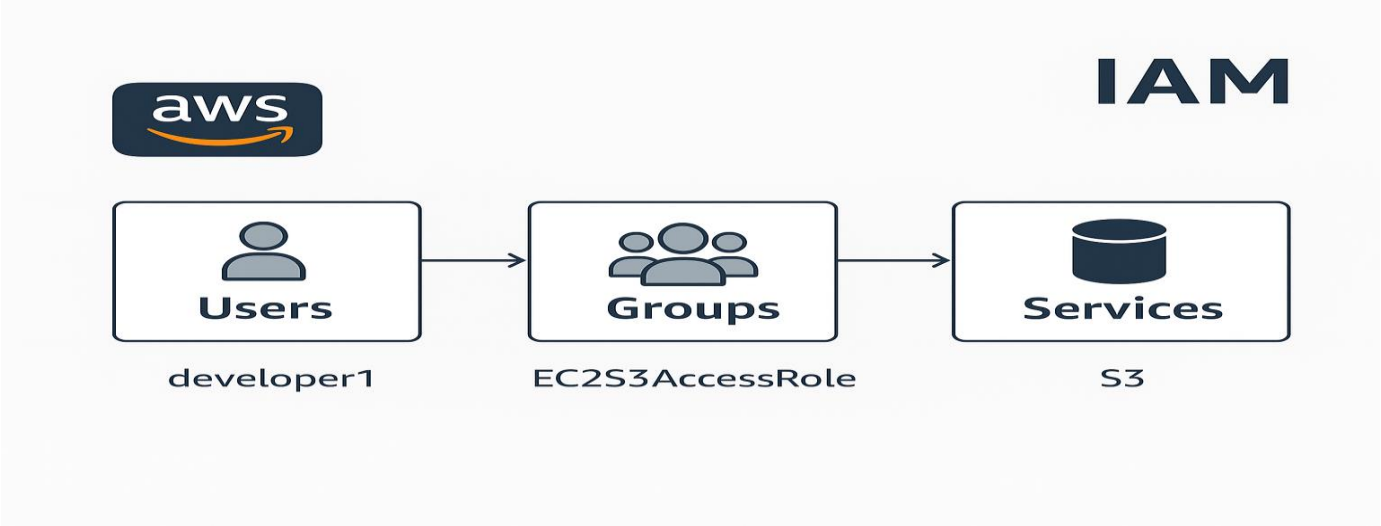
1. Create a role EC2S3AccessRole.
2. Attach AmazonS3ReadOnlyAccess.

3. Launch EC2 instance and attach this role.
   ✅ Now EC2 can access S3 without keys.

Step 8: IAM Best Practices

- ✔ Don’t use Root account.
- ✔ Use groups instead of attaching policies to each user.
- ✔ Apply least privilege (give only required permissions).
- ✔ Enable MFA on all accounts.
- ✔ Rotate credentials regularly.
- ✔ Monitor IAM activity via CloudTrail.

Crate a diagram **(IAM users → groups → roles → services)** so it’s more visual?



AWS Transit Gateway (TGW) — Complete Detailed Explanation(Same Region VPC’s)

✅What is AWS Transit Gateway?

- **AWS Transit Gateway (TGW)** is a **highly scalable, fully managed network hub** that allows you to connect:
  - **Multiple VPCs**
  - **On-premises data centers (via VPN or Direct Connect)**
  - **AWS accounts (multi-account setup)**

TGW acts as a **central router** to simplify complex VPC-to-VPC communication.

Why Transit Gateway Instead of VPC Peering?

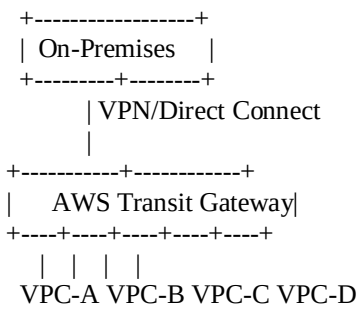
Feature	VPC Peering	Transit Gateway (TGW)

Feature	VPC Peering	Transit Gateway (TGW)
Type	1-to-1	1-to-many / many-to-many
Scaling	Becomes complex with many VPCs	Scales to 1000s of VPCs easily
Transitive Routing	❌Not supported	✅Supported
Centralized Management	❌No	✅Yes
Cost Efficiency (Large)	❌More expensive with more peerings	✅More cost-effective at large scale

How AWS Transit Gateway Works:

- TGW connects to multiple **VPCs** and **on-premises networks** via **attachments**.
- TGW handles **routing between attached networks** internally.
- VPCs don't need to peer with each other directly; they just connect to the TGW.

Architecture Example:



All VPCs can communicate via TGW.

Transit Gateway Attachments:

- **VPC Attachment:** Connect a VPC to the TGW.
- **VPN Attachment:** Connect on-premises to TGW over VPN.
- **Direct Connect Gateway Attachment:** Use AWS Direct Connect.
- **Peering Attachment:** Connect TGWs from different AWS regions.

Transit Gateway Route Table:

- TGW uses its own **route table** to control how traffic flows between attachments.
- Similar to VPC route tables but applies at the TGW level.

→ You can have **multiple TGW route tables** for isolation, e.g., **prod, dev, test**

Real-Time Example Scenario:

Environment	VPC CIDR
VPC-Dev	10.0.0.0/16
VPC-Test	10.1.0.0/16
VPC-Prod	10.2.0.0/16
On-Premises	192.168.1.0/24

All connected via **Transit Gateway**.

- **Dev ↔ Test ↔ Prod ↔ On-Premises** all communicate securely.

Cost Model:

- **Per Attachment/Hour:** You pay for each VPC/VPN attached.
- **Data Processing Cost:** Per GB of data processed by the TGW.

→ Cheaper than thousands of VPC peerings.

Benefits of AWS Transit Gateway:

- ✔ **Scalability:** Connect 1000s of VPCs.
- ✔ **Simplified Architecture:** No complex mesh of peerings.
- ✔ **Centralized Routing:** Control traffic easily.
- ✔ **Transitive Routing Supported.**
- ✔ **Multi-Region:** Use **TGW Peering** for cross-region setups.

Limitations:

- Not free; additional costs.
- CIDR ranges must still not overlap.
- Maximum bandwidth for VPN attachment is capped unless using Direct Connect

Transit Gateway Detailed Hands-On Explanation

Architecture Example:

VPC Name	CIDR	Purpose
VPC-Dev	10.0.0.0/16	Development
VPC-Test	10.1.0.0/16	Testing
VPC-Prod	10.2.0.0/16	Production

Step 1: Create Transit Gateway (TGW)

✅How:

1. Open AWS Console → **VPC** → **Transit Gateways**.
2. Click **"Create Transit Gateway"**.
3. Fill in details:
    - **Name:** e.g., My-TGW.
    - **ASN (Autonomous System Number):** Leave default or set if connecting to on-prem (example: 64512).
    - **Default Route Table Association:** ✔ (optional but recommended).
    - **Multicast Support:** ✖unless needed.
    - **DNS Support:** ✔ (for hostname resolution between VPCs).
4. Click **Create Transit Gateway**.

→ ✅TGW is now created. This is like a virtual router.

Step 2: Attach VPCs to TGW

✅For Each VPC (Dev, Test, Prod):

1. Go to **Transit Gateway Attachments** → **Create Attachment**.
2. Select:
    - **Transit Gateway:** My-TGW.
    - **Attachment Type:** VPC.
    - **VPC:** Choose your VPC (e.g., VPC-Dev).
    - **Subnets:** Select **one subnet per Availability Zone (AZ)** (this is for TGW ENIs).
3. Click **Create Attachment**.

→ Do this for **VPC-Test** and **VPC-Prod** similarly.

Step 3: Configure Transit Gateway Route Table

- Go to **Transit Gateway Route Tables**.
- TGW usually creates a **default route table**, or create separate ones if you want isolation between environments.

✅Add Routes:

- **Destination:** CIDR of the peer VPC.
- **Target:** The TGW Attachment for that VPC.

Step 4: Update VPC Route Tables

In VPC-Dev's Route Table:

- **Destination:** 10.1.0.0/16 → **Target:** TGW-ID.
- **Destination:** 10.2.0.0/16 → **Target:** TGW-ID.

Similarly, update VPC-Test **and** VPC-Prod**:**

- Any traffic destined to the other VPCs goes via **Transit Gateway**.

→ This is **critical**. Without this, packets won't leave the VPC towards the TGW.

Step 5: Update Security Groups

Security Groups must allow traffic between VPCs.

Example (VPC-Dev EC2):

- Inbound rule:
  - **Type:** SSH (or HTTP/DB as needed)
  - **Source:** 10.1.0.0/16 (VPC-Test)
  - Or **Source:** 10.0.0.0/8 (if fully open within TGW-connected VPCs)

Do the same for **Test** and **Prod** VPCs.

Step 6: Verify Communication

→ From an EC2 in **VPC-Dev**, try:

bash
Copy code
ping 10.1.1.10   # EC2 in VPC-Test
ssh ec2-user@10.1.1.10

→ Similarly from VPC-Test to VPC-Prod.

→ ✅This confirms that TGW is routing traffic correctly.

Summary Workflow:

1️⃣ Create TGW →
2️⃣ Attach VPCs →
3️⃣ Update TGW Route Table →
4️⃣ Update VPC Route Tables →
5️⃣ Allow Security Groups →
6️⃣ ✅EC2 ↔ EC2 communication works across VPCs.

Transit Gateway with Inter-Region Peering – Notes

□ What is AWS Transit Gateway?

- A **network transit hub** that connects multiple VPCs and on-premises networks via a single gateway.
- Simplifies VPC-to-VPC and VPC-to-on-premises communication.
- Scales better than VPC Peering (which is point-to-point).

□ What is Transit Gateway Peering?

- **Transit Gateway Peering** allows communication between **Transit Gateways in different AWS regions**.
- VPCs attached to one TGW can communicate with VPCs attached to a peered TGW in another region.

□ Key Features of TGW Peering

- Supports **Inter-region** communication.
- **Traffic is encrypted** and flows over the AWS backbone.
- **No transitive routing** between VPCs in different TGWs (you must attach VPCs directly to their TGW).
- Lower latency and better security compared to VPN or public internet routing.

✅Step-by-Step Process

□ Step 1: Create VPCs in Both Regions

Region	VPC Name	CIDR Block
Mumbai (ap-south-1)	VPC-Mumbai	10.0.0.0/16
Virginia (us-east-1)	VPC-Virginia	192.168.0.0/16

□ Step 2: Create Transit Gateways (TGWs)

- Create TGW in Mumbai → TGW-Mumbai
- Create TGW in Virginia → TGW-Virginia

□ Step 3: Attach VPCs to TGWs

- Attach **VPC-Mumbai** to **TGW-Mumbai**.
- Attach **VPC-Virginia** to **TGW-Virginia**.

□ Step 4: Create TGW Peering Connection

- In **TGW-Mumbai**:
  - Create Peering.
  - Select destination region (**us-east-1 - Virginia**).
  - Enter **TGW ID** of **TGW-Virginia**.
- In **TGW-Virginia**:
  - Accept the peering request.

□ Step 5: Update Transit Gateway Route Tables -→ Routes

- In **TGW-Mumbai Route Table** (Add Another VPC"S CIDR)
  - Route **192.168.0.0/16** → via **Peering Attachment**.
- In **TGW-Virginia Route Table**:
  - Route **10.0.0.0/16** → via **Peering Attachment**.

□ Step 6: Update VPC Route Tables

- In **VPC-Mumbai Route Table**:
  - Route **192.168.0.0/16** → via **TGW Attachment (TGW-Mumbai)**.
- In **VPC-Virginia Route Table**:
  - Route **10.0.0.0/16** → via **TGW Attachment (TGW-Virginia)**.

□ Step 7: Update Security Groups (SG) and NACL

- Allow inbound and outbound traffic between VPC CIDRs.
- Example (for Mumbai):
  - Allow inbound from **192.168.0.0/16** on relevant ports (e.g., SSH, HTTP).
  - Same for outbound.

□ Step 8: Testing the Connection

- Launch EC2 in both VPCs.
- SSH/Ping private IPs between Mumbai and Virginia EC2 instances.
- ✅If set correctly, they communicate privately over AWS backbone.

□ Benefits of TGW Peering Over VPN or VPC Peering

- **Higher performance** (lower latency, AWS backbone).
- **More secure** (no public internet).
- Handles **hundreds of VPCs**, unlike VPC peering which is point-to-point.
- **Easier to manage** complex architectures.

□ Limitations

- No transitive routing between TGWs.
- Cannot use TGW Peering for VPCs attached to a different TGW (unless they have their own attachments).

□ Important Points to Remember

- Each region needs its own TGW.
- TGW attachments are regional.
- Peering enables communication across regions, but you still need to configure:
  - TGW route tables.
  - VPC route tables.
  - Security groups and NACLs.

Conclusion:

- **TGW = YES transitive routing within attached VPCs but VPC's In Same Region.**
- **TGW Peering = NO transitive routing across TGWs Usefull when VPC'S in Different Region.**
- **VPC Peering = Does not Support Transitive routing**

Tansitive touting :- VPC1 <->VPC2 <-->VPC3 but not VPC1<!=>VPC3

VPN (Virtual Private Network)

✅What is VPN?

- VPN creates a **secure, encrypted connection (tunnel)** between:
  - **Your on-premises network (office/data center) AWS VPC**
  - OR between two AWS VPCs (VPC-to-VPC VPN)

✅Why VPN?

- To access AWS resources (EC2, RDS, etc.) **privately and securely** without using the public internet.
- Data is encrypted while traveling over the internet.

How It Works:

- AWS has **Virtual Private Gateway (VGW)** on VPC side.
- On-premises has **Customer Gateway (CGW)** — either hardware (router) or software.
- They connect via a **VPN Tunnel (IPSec)**.

Example:

- Your company office (192.168.1.0/24) → wants to access EC2 in VPC (10.0.0.0/16).
- VPN Tunnel created → your office computers can directly SSH or connect to AWS EC2 via **private IP**.

Key Benefit:

- Secure, private access to AWS without exposing to the internet.

🚀 VPC Endpoints

✅What is a VPC Endpoint?

- A **private connection from your VPC to AWS services (like S3, DynamoDB, etc.) without using the internet.**

→ No Internet Gateway (IGW), No NAT Gateway needed.

✅Why VPC Endpoints?

- More secure — traffic stays inside AWS private network.
- Lower latency, no internet dependency.

Types of VPC Endpoints:

1. **Interface Endpoint (Most AWS services)**
    - Creates an **Elastic Network Interface (ENI)** with a private IP in your subnet.
    - Example: Connect to **SNS, SQS, CloudWatch, API Gateway, etc.** privately.
2. **Gateway Endpoint (Only for S3 & DynamoDB)**
    - Adds an entry in the VPC Route Table to route traffic to **S3/DynamoDB privately.**

□ Example:

- You have EC2 in **private subnet**.
- Normally, EC2 needs a NAT Gateway to access **S3**.
- With **VPC Gateway Endpoint**, EC2 can access S3 directly over AWS backbone — **no internet, no NAT charges**.

□ Key Benefit:

- Cost-saving (no NAT), secure (no internet exposure).

AWS S3 In Detail

□ What is AWS S3?

- **AWS S3** stands for **Simple Storage Service**.
- It is an **object storage service**, meaning it stores data as **objects** (files + metadata).
- It is designed for **scalability, high availability, durability (99.999999999% — 11 9's)**.
- Can store **any amount of data** from anywhere on the web.

S3 Basics Terminology

Term	Description
Bucket	A container for storing objects (like a folder). Unique globally.
Object	Files stored in S3. Includes **data + metadata**.
Key	Unique name of the object inside a bucket (acts like file path).
Value	The actual content of the object (file data).
Metadata	Data about the object (file type, encryption, etc.).
Versioning	Maintains multiple versions of an object.

S3 Use Cases

- Website hosting (Static websites).
- Backup and disaster recovery.
- Big data analytics (input/output data storage).
- Media hosting (images, videos).
- Software delivery (patches, binaries).
- Machine learning data lakes.

Static Website Hosting using AWS S3

❓ What is Static Website Hosting?

- A **static website** delivers **HTML, CSS, JavaScript, images, videos**, etc.
- No server-side processing (no PHP, Node.js, Python, etc.).
- S3 serves static content directly over HTTP/HTTPS without servers (EC2/Lambda).

Key Benefits of Hosting on S3

- Highly available (99.99%).
- Durable (11 9's durability).
- Cost-effective — pay only for storage + bandwidth.
- Scales automatically.
- No server management.

Step 1: Create an S3 Bucket

- Go to **S3 Console**.
- Click **"Create bucket"**.
- Name the bucket (recommended: same as your domain, e.g., `example.com`).
- **Region:** Select desired AWS region.
- **Uncheck**: "Block all public access" (mandatory for website hosting).
- Acknowledge the warning ✅.
- Click **"Create bucket"**.

Step 2: Upload Website Files

- Upload your:
  - `index.html` (Homepage)
  - `error.html` (Optional error page)
  - CSS, JS, images, etc.

Step 3: Configure Bucket for Static Website Hosting

- Go to the bucket.
- Click **"Properties"** tab.
- Scroll to **"Static website hosting"**.
- Enable it.
- Select **"Host a static website"**.
- Set:
  - **Index document:** `index.html`
  - **Error document:** `error.html` (optional)
- Note the **Endpoint URL**:

  ```
  http://<bucket-name>.s3-website-<region>.amazonaws.com
  ```

- This is your website URL

Step 4: Make Content Public (Permissions)

- Go to the **“Permissions”** tab of the bucket.
- Add a **Bucket Policy** to allow public read access.

Example Bucket Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket-name/*"
    }
  ]
}
```

 Replace `your-bucket-name` with your actual bucket name.

- Click **Save changes**.

How To Create a New Bucket Policy

Step 1: Open Policy Generator Tool

Step 2: Select Policy Type

- From **"Select Type of Policy to Generate"** dropdown:
  - Select **“S3 Bucket Policy”** if making for S3.
  - Or select **“IAM Policy”** if it's for IAM, EC2, etc.

Fill these fields:

Field	Description
Effect	`Allow` or `deny`. Usually `Allow` for access.
AWS Service	Select e.g., `Amazon S3`
Actions	Choose permissions (e.g., `Get Object`, `Put Object`).
ARN	Specify the resource. Example for bucket: `arn: aws:s3:::your-bucket-name/*`
Principles:-	*** - Means Public Access**

Step 3: Add Statement And Generate Policy

Step 4: Test Your Website

- Open the **S3 Website Endpoint URL** in a browser:
- http://<bucket-name>.s3-website-<region>.amazonaws.com

Optional – Connect Domain Name

Steps to connect your custom domain (e.g., `nileshbhurewar.com`):

1. **Register Domain** (via Route 53 or any registrar like GoDaddy).

2. Use **Route 53** or update DNS records in your registrar:
    - Create a **CNAME** pointing to:

      ```
      <bucket-name>.s3-website-<region>.amazonaws.com
      ```

3. Or set up with **CloudFront** (recommended for HTTPS).

HTTPS – How to Secure (Optional but Recommended)

S3's website endpoint supports **HTTP only**, not HTTPS directly.

✅**Solution:** Use **CloudFront (CDN)** with an SSL certificate.

Steps:

1. Create **CloudFront distribution**:
    - Origin: Your S3 bucket website endpoint.
    - Enable **Redirect HTTP to HTTPS**.
2. Use **AWS Certificate Manager (ACM)** to get an SSL certificate (free).
3. Update your domain's DNS to point to CloudFront.
